1. Исследование надежности заемщиков.

Заказчик — кредитный отдел банка. Нужно разобраться, влияет ли семейное положение и количество детей клиента на факт погашения кредита в срок. Входные данные от банка — статистика о платёжеспособности клиентов. Результаты исследования будут учтены при построении модели кредитного скоринга — специальной системы, которая оценивает способность потенциального заёмщика вернуть кредит банку.

Имеющиеся признаки:

- children — количество детей в семье
- days_employed — общий трудовой стаж в днях
- dob_years — возраст клиента в годах
- education — уровень образования клиента
- education_id — идентификатор уровня образования
- family_status — семейное положение
- family_status_id — идентификатор семейного положения
- gender — пол клиента
- income_type — тип занятости
- debt — имел ли задолженность по возврату кредитов
- total_income — ежемесячный доход
- purpose — цель получения кредита

1.0.2. Шаг 1. Обзор данных

Ввод [2]:

```
import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

Ввод [4]:

```
data = pd.read_csv('/datasets/data.csv')
display(data.head(10))
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1
5	0	-926.185831	27	высшее	0	гражданский брак	1
6	0	-2879.202052	43	высшее	0	женат / замужем	0
7	0	-152.779569	50	СРЕДНЕЕ	1	женат / замужем	0
8	2	-6929.865299	35	ВЫСШЕЕ	0	гражданский брак	1
9	0	-2188.756445	41	среднее	1	женат / замужем	0

Рассмотрим данные чуть подробнее

Ввод [5]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21525 entries, 0 to 21524
Data columns (total 12 columns):
children            21525 non-null int64
days_employed       19351 non-null float64
dob_years           21525 non-null int64
education           21525 non-null object
education_id        21525 non-null int64
family_status       21525 non-null object
family_status_id    21525 non-null int64
gender              21525 non-null object
income_type         21525 non-null object
debt                21525 non-null int64
total_income        19351 non-null float64
purpose             21525 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 2.0+ MB
```

days_employed и total_income имеют пропуски, причем одинаковое количество

Ввод [6]:

```
days_employed_null = data['days_employed'].isnull().sum()
days_employed_total = len(data['days_employed'])
print(f'Пропусков в days_employed {round(days_employed_null / days_employed_total *
```

```
Пропусков в days_employed 10.1%
```

Ввод [7]:

```
total_income_null = data['total_income'].isnull().sum()
total_income_total = len(data['total_income'])
print(f'Пропусков в total_income {round(total_income_null / total_income_total * 100
```

```
Пропусков в total_income 10.1%
```

Ввод [8]:

```
display(data[data.days_employed.isnull()].head())
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
12	0	NaN	65	среднее	1	гражданский брак	1
26	0	NaN	41	среднее	1	женат / замужем	0
29	0	NaN	63	среднее	1	Не женат / не замужем	4
41	0	NaN	50	среднее	1	женат / замужем	0
55	0	NaN	54	среднее	1	гражданский брак	1

Ввод [9]:

```
display(data[data.total_income.isnull()].tail())
```

	children	days_employed	dob_years	education	education_id	family_status	family_status
21489	2	NaN	47	Среднее	1	женат / замужем	
21495	1	NaN	50	среднее	1	гражданский брак	
21497	0	NaN	48	ВЫСШЕЕ	0	женат / замужем	
21502	1	NaN	42	среднее	1	женат / замужем	
21510	2	NaN	28	среднее	1	женат / замужем	

Судя по всему, некоторые не указывают стаж работы, отсюда пропуски в days_employed

Есть вероятность, что пропуски в одних и тех же строках: логично, что у неработающего человека будет отсутствовать доход

Ввод [10]:

```
# сравним индексы строк с пропущенными значениями в обоих столбцах
list(data[data.days_employed.isnull()].index) == list(data[data.total_income.isnull(
```

Out[10]:

```
True
```

Действительно, пропуски встречаются только вместе

Было бы неплохо удалить их, так как с данными о доходе, не зная стажа работы, работать трудно - его может не быть вовсе, а может быть очень большой, в обход налогов

Но пропусков 10%, мы себе такую роскошь позволить не можем, следовательно, заполняем медианными значениями (также, можно еще ориентироваться на сферу деятельности)

1.0.3. Шаг 2.1 Заполнение пропусков

Ввод [12]:

```
# Создаем функцию заполнения пропусков по сферам деятельности
def income_sphere(row):
    if pd.isna(row['total_income']):
        return data[data.income_type == row.income_type]['total_income'].median()
    else:
        return row['total_income']
```

Ввод [13]:

```
data['days_employed'] = data['days_employed'].fillna(data['days_employed'].median())
data['total_income'] = data.apply(income_sphere, axis=1)
display(data.days_employed.isna().sum())
display(data.total_income.isna().sum())
```

```
0

0
```

Ввод [14]:

```
display(data.head())
display(data.tail())
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1

	children	days_employed	dob_years	education	education_id	family_status	family_status
21520	1	-4529.316663	43	среднее	1	гражданский брак	
21521	0	343937.404131	67	среднее	1	женат / замужем	
21522	1	-2113.346888	38	среднее	1	гражданский брак	
21523	3	-3112.481705	38	среднее	1	женат / замужем	
21524	2	-1984.507589	40	среднее	1	женат / замужем	

Вышли отрицательные значения, с ними и будем разбираться

1.0.4. Шаг 2.2 Проверка данных на аномалии и исправления.

1.0.4.1. Общая проверка

Посмотрим на основные данные, категориальные, где можно оценить ситуацию на глаз

Ввод [15]:

```
data_categorical_list = ['children', 'education', 'education_id', 'family_status',
                         'family_status_id', 'gender', 'income_type', 'debt', 'purpo

for categoria in data_categorical_list:
    print(categoria)
    print(data[categoria].unique())
    print()
```

```
children
[ 1  0  3  2 -1  4 20  5]

education
['высшее' 'среднее' 'Среднее' 'СРЕДНЕЕ' 'ВЫСШЕЕ' 'неоконченное высшее'
 'начальное' 'Высшее' 'НЕОКОНЧЕННОЕ ВЫСШЕЕ' 'Неоконченное высшее'
 'НАЧАЛЬНОЕ' 'Начальное' 'Ученая степень' 'УЧЕНАЯ СТЕПЕНЬ'
 'ученая степень']

education_id
[0 1 2 3 4]

family_status
['женат / замужем' 'гражданский брак' 'вдовец / вдова' 'в разводе'
 'Не женат / не замужем']

family_status_id
[0 1 2 3 4]

gender
['F' 'M' 'XNA']

income_type
['сотрудник' 'пенсионер' 'компаньон' 'госслужащий' 'безработный'
 'предприниматель' 'студент' 'в декрете']

debt
[0 1]

purpose
['покупка жилья' 'приобретение автомобиля' 'дополнительное образовани
е'
 'сыграть свадьбу' 'операции с жильем' 'образование'
 'на проведение свадьбы' 'покупка жилья для семьи' 'покупка недвижимос
ти'
 'покупка коммерческой недвижимости' 'покупка жилой недвижимости'
 'строительство собственной недвижимости' 'недвижимость'
 'строительство недвижимости' 'на покупку подержанного автомобиля'
 'на покупку своего автомобиля' 'операции с коммерческой недвижимость
ю'
 'строительство жилой недвижимости' 'жилье'
 'операции со своей недвижимостью' 'автомобили' 'заняться образование
м'
 'сделка с подержанным автомобилем' 'получение образования' 'автомобил
ь'
 'свадьба' 'получение дополнительного образования' 'покупка своего жил
ья'
 'операции с недвижимостью' 'получение высшего образования'
 'свой автомобиль' 'сделка с автомобилем' 'профильное образование'
 'высшее образование' 'покупка жилья для сдачи' 'на покупку автомобил
я'
```

'ремонт жилью' 'заняться высшим образованием']

Очень хочется посмотреть на семью счастливых XNA с -1 ребенком, но имеем что имеем

1.0.4.2. Дети и гендер

Оценим масштаб ошибок с детьми и полом

Ввод [16]:

```
anomal_rows = {'children': [-1, 20],
               'gender': ['XNA']}

for row in anomal_rows:
    print(row)
    print()
    for value in anomal_rows[row]:
        value_count = len(data[data[row] == value])
        print(f'{value}: {value_count} ({round(value_count / len(data[row]) * 100, 2
    print()
    print()
```

```
children

-1: 47 (0.22%)
20: 76 (0.35%)


gender

XNA: 1 (0.0%)
```

Можно предположить, что -1 ребенок появился из 1 ребенка, а 20 – из 2

Но так как их сравнительно незначительное количество, проще -1 заменить на 1, а 20 – заменить медианой, округленной вниз

Ввод [17]:

```
# Заменяем -1 детей на 1 детей
data['children'] = data['children'].replace(-1, 1)
print('-1 детей:', len(data[data['children'] == -1]))

# Заменяем 20 детей на mean детей
data['children'] = data['children'].replace(20, int(data['children'].mean()))
print('20 детей:', len(data[data['children'] == 20]))
```

```
-1 детей: 0
20 детей: 0
```

С неизвестным гендером работаем культурно и толерантно - убираем из выборки

Ввод [18]:

```
# Запомним сколько было
gender_count_old = len(data['gender'])
data = data[data['gender'] != 'XNA']
# Метод костыльный, так что перестраховываемся
gender_count_new = len(data['gender'])
print('Удалили гендеров:', gender_count_old-gender_count_new)
```

```
Удалили гендеров: 1
```

Сверяемся:

Ввод [19]:

```
data_categorical_list = ['children', 'gender']

for categoria in data_categorical_list:
    print(categoria)
    print(data[categoria].unique())
    print()
```

```
children
[1 0 3 2 4 5]

gender
['F' 'M']
```

1.0.4.3. Образование

Разберемся с образованием: там все проще, у нас явные дубликаты из-за регистра - приведем все к нижнему

Ввод [20]:

```
data['education_lower'] = data['education'].str.lower()
display(data.head(10))
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1
5	0	-926.185831	27	высшее	0	гражданский брак	1
6	0	-2879.202052	43	высшее	0	женат / замужем	0
7	0	-152.779569	50	СРЕДНЕЕ	1	женат / замужем	0
8	2	-6929.865299	35	ВЫСШЕЕ	0	гражданский брак	1
9	0	-2188.756445	41	среднее	1	женат / замужем	0

Проверим новый столбец

Ввод [21]:

```
display(data['education_lower'].unique())
```

```
array(['высшее', 'среднее', 'неоконченное высшее', 'начальное',
       'ученая степень'], dtype=object)
```

1.0.5. Шаг 2.7. Категоризация целей кредита.

1.0.5.1. Цель кредита

Теперь разберемся с целью кредита

Ввод [22]:

```
for row in data['purpose'].unique():
    print(row)
```

```
покупка жилья
приобретение автомобиля
дополнительное образование
сыграть свадьбу
операции с жильем
образование
на проведение свадьбы
покупка жилья для семьи
покупка недвижимости
покупка коммерческой недвижимости
покупка жилой недвижимости
строительство собственной недвижимости
недвижимость
строительство недвижимости
на покупку подержанного автомобиля
на покупку своего автомобиля
операции с коммерческой недвижимостью
строительство жилой недвижимости
жилье
операции со своей недвижимостью
автомобили
заняться образованием
сделка с подержанным автомобилем
получение образования
автомобиль
свадьба
получение дополнительного образования
покупка своего жилья
операции с недвижимостью
получение высшего образования
свой автомобиль
сделка с автомобилем
профильное образование
высшее образование
покупка жилья для сдачи
на покупку автомобиля
ремонт жилью
заняться высшим образованием
```

Попробуем хоть как то автоматизировать процесс

Ввод [23]:

```
# Создаем функциб с подстановкой значений, которые выделили на глаз (куски основных

def purpose_dict_pred(row):
    purposes = {
        'жил': 'недвижимость',
        'авто': 'автомобили',
        'образов': 'образование',
        'недвиж': 'недвижимость',
        'свад': 'свадьба'
    }
    for word in purposes:
        if word in row['purpose']:
            return purposes[word]
```

1.0.5.2. Добавление столбцов

Дополняем и объединяем категории

Ввод [24]:

```
data['purpose_total'] = data.apply(purpose_dict_pred, axis=1)
display(data.head(10))
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1
5	0	-926.185831	27	высшее	0	гражданский брак	1
6	0	-2879.202052	43	высшее	0	женат / замужем	0
7	0	-152.779569	50	СРЕДНЕЕ	1	женат / замужем	0
8	2	-6929.865299	35	ВЫСШЕЕ	0	гражданский брак	1
9	0	-2188.756445	41	среднее	1	женат / замужем	0

Ну и раз уж мы здесь, приведем к божескому виду еще и гендеры со статусами

Ввод [25]:

```
data['gender_lower'] = data['gender'].str.lower()
data['family_status_lower'] = data['family_status'].str.lower()
display(data.head())
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1

Проведем повторную проверку данных

Ввод [26]:

```
data_categorical_list = ['children', 'education_lower', 'family_status_lower',
                         'gender_lower', 'purpose_total']

for categoria in data_categorical_list:
    print(categoria)
    print(data[categoria].unique())
    print()
```

```
children
[1 0 3 2 4 5]

education_lower
['высшее' 'среднее' 'неоконченное высшее' 'начальное' 'ученая степен
ь']

family_status_lower
['женат / замужем' 'гражданский брак' 'вдовец / вдова' 'в разводе'
 'не женат / не замужем']

gender_lower
['f' 'm']

purpose_total
['недвижимость' 'автомобили' 'образование' 'свадьба']
```

1.0.5.3. Числовые значения – общая проверка

Переходим к числовым значениям, проверим их на адекватность

Ввод [27]:

```
digit_values = ['days_employed', 'dob_years', 'total_income', 'dob_years']
for column in digit_values:
    print(column)
    print('Минимум:', data[column].min())
    print('Максимум', data[column].max())
    print('Среднее', data[column].mean())
    print('Медиана', data[column].median())
    print('Ошибок:', data[column].isna().sum())
    print()

data[['days_employed', 'dob_years', 'total_income']].info()
```

```
days_employed
Минимум: -18388.949900568383
Максимум 401755.40047533
Среднее 56560.0729182367
Медиана -1203.369528770489
Ошибок: 0

dob_years
Минимум: 0
Максимум 75
Среднее 43.29427615684817
Медиана 42.0
Ошибок: 0

total_income
Минимум: 20667.26379327158
Максимум 2265604.028722744
Среднее 165223.5274577049
Медиана 142594.39684740017
Ошибок: 0

dob_years
Минимум: 0
Максимум 75
Среднее 43.29427615684817
Медиана 42.0
Ошибок: 0

<class 'pandas.core.frame.DataFrame'>
Int64Index: 21524 entries, 0 to 21524
Data columns (total 3 columns):
days_employed    21524 non-null float64
dob_years        21524 non-null int64
total_income     21524 non-null float64
dtypes: float64(2), int64(1)
memory usage: 672.6 KB
```

Возраст клиентов выглядит не особо прилично, надо с ним разобраться

1.0.5.4. Возраст клиентов

Выведем все возрасты по возрастанию (неожиданная тавтология)

Ввод [28]:

```
print(sorted(list(data['dob_years'].unique())))
```

```
[0, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 3
5, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52,
53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 7
0, 71, 72, 73, 74, 75]
```

Узнаем, сколько у нас 0-летних заемщиков

Ввод [29]:

```
print(len(data[data.dob_years == 0]))
```

```
101
```

Немало. Но тут ничего нельзя поделать, узнать, сколько им реально лет не получится, то ли 20, то ли 50... Игнорируем!

1.0.5.5. Изменение days_employed

Хотел бы я работать отрицательное количество дней, но раз у меня нет такой возможности, лишим ее всех

Отрицательные значения заменим на положительные, скорее всего ошибка выгрузки, хотя и не факт, но обратиться к грузчикам возможности нет, работаем с тем что есть

Ввод [30]:

```
# как обычно - спасает apply, int спокойно вернет отрицательное
data['days_employed_integer'] = data['days_employed'].apply(abs)
display(data.head())
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
0	1	-8437.673028	42	высшее	0	женат / замужем	0
1	1	-4024.803754	36	среднее	1	женат / замужем	0
2	0	-5623.422610	33	Среднее	1	женат / замужем	0
3	3	-4124.747207	32	среднее	1	женат / замужем	0
4	0	340266.072047	53	среднее	1	гражданский брак	1

Похоже, тут все неплохо, посмотрим снова

Ввод [31]:

```
column = 'days_employed_integer'
print(column)
print('Минимум:', data[column].min())
print('Максимум', data[column].max())
print('Среднее', data[column].mean())
print('Медиана', data[column].median())
```

```
days_employed_integer
Минимум: 24.14163324048118
Максимум 401755.40047533
Среднее 60280.6488492364
Медиана 1807.8486042866098
```

Ввод [32]:

```
print("401755 дней в годах это ~", int(401755 / 365), 'лет')
```

```
401755 дней в годах это ~ 1100 лет
```

Ввод [33]:

```
display(data.sort_values(by='days_employed_integer').tail(10))
```

	children	days_employed	dob_years	education	education_id	family_status	family_status
8369	0	401590.452231	58	среднее	1	женат / замужем	
10991	0	401591.828457	56	среднее	1	в разводе	
17823	0	401614.475622	59	среднее	1	женат / замужем	
13420	0	401619.633298	63	Среднее	1	гражданский брак	
4697	0	401635.032697	56	среднее	1	женат / замужем	
7794	0	401663.850046	61	среднее	1	гражданский брак	
2156	0	401674.466633	60	среднее	1	женат / замужем	
7664	1	401675.093434	61	среднее	1	женат / замужем	
10006	0	401715.811749	69	высшее	0	Не женат / не замужем	
6954	0	401755.400475	56	среднее	1	вдовец / вдова	

И такая ситуация не одна, людей буквально держат в вечном рабстве и пора это исправить

Есть подозрение, что 1100 лет это на самом деле 11 лет, но мы попробуем исхитриться и оставить только тех, кто работает дольше своей жизни после 18 лет

Ввод [34]:

```
data_zombie = data[data['days_employed_integer'] / 365 > data['dob_years']]
print(len(data_zombie['dob_years']))
display(data_zombie.head(10))
```

```
3529
```

	children	days_employed	dob_years	education	education_id	family_status	family_status_id
4	0	340266.072047	53	среднее	1	гражданский брак	1
18	0	400281.136913	53	среднее	1	вдовец / вдова	2
24	1	338551.952911	57	среднее	1	Не женат / не замужем	4
25	0	363548.489348	67	среднее	1	женат / замужем	0
30	1	335581.668515	62	среднее	1	женат / замужем	0
35	0	394021.072184	68	среднее	1	гражданский брак	1
50	0	353731.432338	63	среднее	1	женат / замужем	0
56	0	370145.087237	64	среднее	1	вдовец / вдова	2
71	0	338113.529892	62	среднее	1	женат / замужем	0
78	0	359722.945074	61	высшее	0	женат / замужем	0

Пять с половиной тысяч несчастных, надо узнать, на сколько они переработали

Ввод [35]:

```
def pererabotka(row):
    return int(row['days_employed_integer'] / 365) - row['dob_years']

hight_work = data_zombie.apply(pererabotka, axis=1)
display(hight_work.head())
```

```
4       879
18     1043
24      870
25      929
30      857
dtype: int64
```

Немало, узначем чуть глубже

Ввод [36]:

```
print('Минимум:', hight_work.min())
print('Максимум', hight_work.max())
print('Среднее', hight_work.mean())
print('Медиана', hight_work.median())
```

```
Минимум: 0
Максимум 1098
Среднее 918.1320487390195
Медиана 937.0
```

В целом, можно стандартно заменить средними значениями

Возвращаем справедливость

Ввод [37]:

```
data_mean = data[data['days_employed_integer'] / 365 < data['dob_years']].mean()
data.loc[(data['days_employed_integer'] / 365 > data['dob_years']), 'days_employed_i

column = 'days_employed_integer'
print(column)
print('Минимум:', data[column].min())
print('Максимум', data[column].max())
print('Среднее', data[column].mean())
print('Медиана', data[column].median())
```

```
days_employed_integer
Минимум: 24.14163324048118
Максимум 18388.949900568383
Среднее 2215.391868119628
Медиана 1347.6449313585415
```

Ввод [38]:

```
print("18388 дней в годах это ~", int(18388 / 365), 'лет')
```

```
18388 дней в годах это ~ 50 лет
```

Ввод [39]:

```
display(data[data.days_employed_integer == 18388.949900568383])
```

	children	days_employed	dob_years	education	education_id	family_status	family_status
16335	1	-18388.949901	61	среднее	1	женат / замужем	

Странно начинать работать в 11 лет, но это уже чуть больше похоже на правду

Ввод [40]:

```
digit_values = ['days_employed_integer', 'dob_years', 'total_income']
for column in digit_values:
    print(column)
    print('Минимум:', data[column].min())
    print('Максимум', data[column].max())
    print('Среднее', data[column].mean())
    print('Медиана', data[column].median())
    print()

data[['days_employed', 'dob_years', 'total_income']].info()
```

```
days_employed_integer
Минимум: 24.14163324048118
Максимум 18388.949900568383
Среднее 2215.391868119628
Медиана 1347.6449313585415

dob_years
Минимум: 0
Максимум 75
Среднее 43.29427615684817
Медиана 42.0

total_income
Минимум: 20667.26379327158
Максимум 2265604.028722744
Среднее 165223.5274577049
Медиана 142594.39684740017

<class 'pandas.core.frame.DataFrame'>
Int64Index: 21524 entries, 0 to 21524
Data columns (total 3 columns):
days_employed    21524 non-null float64
dob_years        21524 non-null int64
total_income     21524 non-null float64
dtypes: float64(2), int64(1)
memory usage: 672.6 KB
```

1.0.6. Шаг 2.3. Изменение типов данных.

Посмотрим какие у нас вообще есть типы данных

Ввод [41]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 21524 entries, 0 to 21524
Data columns (total 17 columns):
children                 21524 non-null int64
days_employed            21524 non-null float64
dob_years                21524 non-null int64
education                21524 non-null object
education_id             21524 non-null int64
family_status            21524 non-null object
family_status_id         21524 non-null int64
gender                   21524 non-null object
income_type              21524 non-null object
debt                     21524 non-null int64
total_income             21524 non-null float64
purpose                  21524 non-null object
education_lower          21524 non-null object
purpose_total            21524 non-null object
gender_lower             21524 non-null object
family_status_lower      21524 non-null object
days_employed_integer    17995 non-null float64
dtypes: float64(3), int64(5), object(9)
memory usage: 3.0+ MB
```

Кстати, многовато столбцов вышло, оставим только необходимые

Ввод [42]:

```
data_cool = data.drop(['days_employed', 'education', 'family_status', 'gender'], axi
```

Ввод [43]:

```
data_cool.head(10)
```

Out[43]:

	children	dob_years	education_id	family_status_id	income_type	debt	total_income	
0	1	42	0	0	сотрудник	0	253875.639453	покуг
1	1	36	1	0	сотрудник	0	112080.014102	прис ав
2	0	33	1	0	сотрудник	0	145885.952297	покуг
3	3	32	1	0	сотрудник	0	267628.550329	дополн обр
4	0	53	1	1	пенсионер	0	158616.077870	
5	0	27	0	1	компаньон	0	255763.565419	покуг
6	0	43	0	0	компаньон	0	240525.971920	ос
7	0	50	1	0	сотрудник	0	135823.934197	обр
8	2	35	0	1	сотрудник	0	95856.832424	на пр
9	0	41	1	0	сотрудник	0	144425.938277	покуг д

Ввод [44]:

```
data_cool.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 21524 entries, 0 to 21524
Data columns (total 13 columns):
children                 21524 non-null int64
dob_years                21524 non-null int64
education_id             21524 non-null int64
family_status_id         21524 non-null int64
income_type              21524 non-null object
debt                     21524 non-null int64
total_income             21524 non-null float64
purpose                  21524 non-null object
education_lower          21524 non-null object
purpose_total            21524 non-null object
gender_lower             21524 non-null object
family_status_lower      21524 non-null object
days_employed_integer    17995 non-null float64
dtypes: float64(2), int64(5), object(6)
memory usage: 2.3+ MB
```

Пора привести total_income, dob_years и days_employed_integer к божескому виду

Ввод [45]:

```
data_cool['total_income'] = data_cool['total_income'].astype('int64')
display(data_cool['total_income'])
```

```
0        253875
1        112080
2        145885
3        267628
4        158616
          ...
21520    224791
21521    155999
21522     89672
21523    244093
21524     82047
Name: total_income, Length: 21524, dtype: int64
```

Ввод [46]:

```
data_cool['dob_years'] = data_cool['dob_years'].astype('int64')
display(data_cool['dob_years'])
```

```
0        42
1        36
2        33
3        32
4        53
         ..
21520    43
21521    67
21522    38
21523    38
21524    40
Name: dob_years, Length: 21524, dtype: int64
```

Ввод [48]:

```
data_cool['days_employed_integer'].isna().sum()
```

Out[48]:

```
3529
```

Заполняем средними значениями

Ввод [49]:

```
fixed_days = data_cool['days_employed_integer'].fillna(data_cool['days_employed_inte
data_cool['days_employed_integer'] = fixed_days
```

Ввод [50]:

```
data_cool['days_employed_integer'].isna().sum()
```

Out[50]:

```
0
```

Ввод [51]:

```
data_cool['days_employed_integer'] = data_cool['days_employed_integer'].astype('int6
display(data_cool['days_employed_integer'])
```

```
0        8437
1        4024
2        5623
3        4124
4        2215
         ...
21520    4529
21521    2215
21522    2113
21523    3112
21524    1984
Name: days_employed_integer, Length: 21524, dtype: int64
```

Ввод [52]:

```
data_cool.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 21524 entries, 0 to 21524
Data columns (total 13 columns):
children                 21524 non-null int64
dob_years                21524 non-null int64
education_id             21524 non-null int64
family_status_id         21524 non-null int64
income_type              21524 non-null object
debt                     21524 non-null int64
total_income             21524 non-null int64
purpose                  21524 non-null object
education_lower          21524 non-null object
purpose_total            21524 non-null object
gender_lower             21524 non-null object
family_status_lower      21524 non-null object
days_employed_integer    21524 non-null int64
dtypes: int64(7), object(6)
memory usage: 2.3+ MB
```

Выглядит культурно и прилично, а значит нам нравится. Оставляем

1.0.8. Шаг 2.5. Формирование дополнительных датафреймов словарей, декомпозиция исходного датафрейма.

Разберемся с делами насущными - с образованием

Ввод [54]:

```
data_education_only = data_cool[['education_lower', 'education_id']].drop_duplicates
display(data_education_only.head())
```

	education_lower	education_id
0	высшее	0
1	среднее	1
2	неоконченное высшее	2
3	начальное	3
4	ученая степень	4

И с делами грядущими - семьей

Ввод [55]:

```
data_family_only = data_cool[['family_status_lower', 'family_status_id']].drop_dupli
display(data_family_only.head())
```

	family_status_lower	family_status_id
0	женат / замужем	0
1	гражданский брак	1
2	вдовец / вдова	2
3	в разводе	3
4	не женат / не замужем	4

Избавляемся от громоздких и ошибкоопасных столбцов в исходном датафрейме

Ввод [56]:

```
data_cool = data_cool.drop(['education_lower', 'family_status_lower'], axis=1)
```

Ввод [57]:

```
data_cool.head()
```

Out[57]:

	children	dob_years	education_id	family_status_id	income_type	debt	total_income	
0	1	42	0	0	сотрудник	0	253875	покупк
1	1	36	1	0	сотрудник	0	112080	приоб авто
2	0	33	1	0	сотрудник	0	145885	покупк
3	3	32	1	0	сотрудник	0	267628	дополни обра
4	0	53	1	1	пенсионер	0	158616	

1.0.9. Шаг 2.6. Категоризация дохода.

Создаем функцию, делаем apply

Ввод [58]:

```
def total_income_categorization(row):
    if row <= 30000:
        return 'E'
    if row <= 50000:
        return 'D'
    if row <= 200000:
        return 'C'
    if row <= 1000000:
        return 'B'
    return 'A'

data_cool['total_income_category'] = data_cool['total_income'].apply(total_income_ca
display(data_cool.head())
```

	children	dob_years	education_id	family_status_id	income_type	debt	total_income	
0	1	42	0	0	сотрудник	0	253875	покупк
1	1	36	1	0	сотрудник	0	112080	приоб авто
2	0	33	1	0	сотрудник	0	145885	покупк
3	3	32	1	0	сотрудник	0	267628	дополни обра
4	0	53	1	1	пенсионер	0	158616	

И вот они уже как на ладони

1.0.10. Пора исследовать

Посмотрим на неплательщиков

Ввод [59]:

```
data_cool[data_cool.debt == 1].head()
```

Out[59]:

	children	dob_years	education_id	family_status_id	income_type	debt	total_income	
14	0	56	0	1	компаньон	1	165127	покуп недви
32	0	34	1	1	сотрудник	1	139057	на пр
38	0	25	0	4	сотрудник	1	192247	обр
55	0	54	1	1	пенсионер	1	118514	
75	1	38	1	0	сотрудник	1	81935	ог недвиж

Посмотрим на каждый параметр

Ввод [60]:

```
data_cool.pivot_table(index=['children'],
                      values=['debt'],
                      aggfunc='mean'
                     ).sort_values(by='debt')
```

Out[60]:

children	debt
5	0.000000
0	0.075295
3	0.081818
1	0.091470
2	0.094404
4	0.097561

Как видно, чаще всего не платят люди с 4 детьми, недалеко от них с 2 и с 1, после чего уже с отрывом идут с 3 и без детей. Абсолютно честные люди - с 5 детьми, скорее всего их просто было не так много

Ввод [61]:

```
data_cool.groupby('children')['debt'].agg(['count','sum','mean'])
```

Out[61]:

children	count	sum	mean
0	14224	1071	0.075295
1	4865	445	0.091470
2	2055	194	0.094404
3	330	27	0.081818
4	41	4	0.097561
5	9	0	0.000000

Ввод [62]:

```
def my_mean(x): return x.mean()*100

# В кач-ве агрегирования можно любую функцию брать.
data_cool.groupby('children')['debt'].agg(['count', 'sum', my_mean])
```

Out[62]:

children	count	sum	my_mean
0	14224	1071	7.529528
1	4865	445	9.146968
2	2055	194	9.440389
3	330	27	8.181818
4	41	4	9.756098
5	9	0	0.000000

Ввод [63]:

```
def my_mean(x): return str(round(x.mean()*100,2)) +'%'

data_cool.groupby('children')['debt'].agg(['count', 'sum', my_mean])
```

Out[63]:

children	count	sum	my_mean
0	14224	1071	7.53%
1	4865	445	9.15%
2	2055	194	9.44%
3	330	27	8.18%
4	41	4	9.76%
5	9	0	0.0%

Ввод [64]:

```
def my_mean(x): return '{:.2%} '.format(x.mean())

data_cool.groupby('children')['debt'].agg(['count', 'sum', my_mean])
```

Out[64]:

children	count	sum	my_mean
0	14224	1071	7.53%
1	4865	445	9.15%
2	2055	194	9.44%
3	330	27	8.18%
4	41	4	9.76%
5	9	0	0.00%

Ввод [65]:

```
data_cool.pivot_table(index=['family_status_id'],
                      values=['debt'],
                      aggfunc='mean'
                     ).sort_values(by='debt').merge(data_family_only, on='family_sta
```

Out[65]:

	family_status_id	debt	family_status_lower
0	2	0.065625	вдовец / вдова
1	3	0.071130	в разводе
2	0	0.075202	женат / замужем
3	1	0.092912	гражданский брак
4	4	0.097405	не женат / не замужем

Самые частые задержки у холостых, самые редкие - у товарищей с браком в прошлом

Ввод [66]:

```
data_cool.pivot_table(index=['total_income_category'],
                      values=['debt'],
                      aggfunc='mean'
                     ).sort_values(by='debt')
```

Out[66]:

total_income_category	debt
D	0.060000
B	0.070621
A	0.080000
C	0.084546
E	0.090909

Можно сделать вывод, что ниже всего задержки у категории D, хотя самые высокие показатели невозврата - у E

Ввод [67]:

```
data_cool.pivot_table(index=['purpose_total'],
                      values=['debt'],
                      aggfunc='mean'
                     ).sort_values(by='debt')
```

Out[67]:

purpose_total	debt
недвижимость	0.072147
свадьба	0.079216
образование	0.091994
автомобили	0.093395

Здесь можно составить рейтинг, который говорит о том, что самые честные - те кто работают с недвижимостью, а вот автомобили оставляют желать лучшего

Ввод [68]:

```
data_cool.pivot_table(index=['purpose'],
                      values=['debt'],
                      aggfunc='mean'
                     ).sort_values(by='debt')
```

Out[68]:

purpose	debt
покупка своего жилья	0.054839
ремонт жилью	0.057190
строительство собственной недвижимости	0.066142
недвижимость	0.066246
покупка жилой недвижимости	0.067545
покупка недвижимости	0.069021
покупка жилья для семьи	0.070203
покупка коммерческой недвижимости	0.070783
жилье	0.071097
образование	0.071588
операции с жильем	0.073507
покупка жилья	0.074189
сыграть свадьбу	0.074935
на покупку подержанного автомобиля	0.075157
строительство жилой недвижимости	0.076677
операции со своей недвижимостью	0.079365
покупка жилья для сдачи	0.079632
операции с коммерческой недвижимостью	0.079877
свадьба	0.080301
операции с недвижимостью	0.081361
дополнительное образование	0.082251
на проведение свадьбы	0.082368
получение образования	0.083521
автомобиль	0.084848
заняться высшим образованием	0.086694
строительство недвижимости	0.087097
высшее образование	0.088300
приобретение автомобиля	0.090909
на покупку своего автомобиля	0.091089
автомобили	0.092050
на покупку автомобиля	0.093220
заняться образованием	0.094660

purpose	debt
свой автомобиль	0.100000
профильное образование	0.100917
сделка с подержанным автомобилем	0.104294
получение высшего образования	0.107981
сделка с автомобилем	0.109890
получение дополнительного образования	0.114094

Здесь видно, что получение дополнительного образования является самым частым кредитом с задержками выплат, и получение высшего тоже недалеко. Однако, сделки с недвижимостью все таки определились верно - у них самые слабые показатели невозврата

1.0.11. Ответы на вопросы.

Вопрос 1:

Есть ли зависимость между количеством детей и возвратом кредита в срок?

Вывод 1:

Да, люди с 4 детьми, 1 ребенком и вообще без детей реже возвращают кредиты

Вопрос 2:

Есть ли зависимость между семейным положением и возвратом кредита в срок?

Вывод 2:

Да, самые частые задержки - у людей, не закрепивших свой брак, а самые редкие - у тех, кто был со своей половинкой в прошлом

Вопрос 3:

Есть ли зависимость между уровнем дохода и возвратом кредита в срок?

Вывод 3:

Есть, однако топ-категории с задержками и без находятся очень рядом друг с другом

Вопрос 4:

Как разные цели кредита влияют на его возврат в срок?

Вывод 4:

С недвижимости возвращают больше кредитов, с образованием и автомобилями - меньше

1.1. Общий вывод:

Каждый параметр влияет на показатели возврата кредита вовремя, портрет самого честного человека - человек в разводе, с 5 детьми и желающий взять кредит на операции с недвижимостью